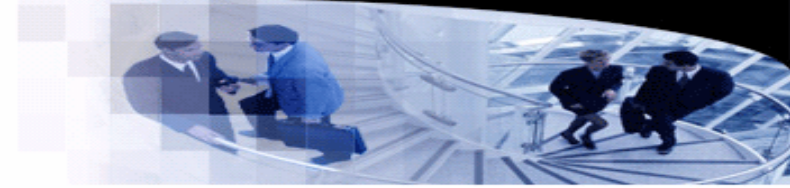




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

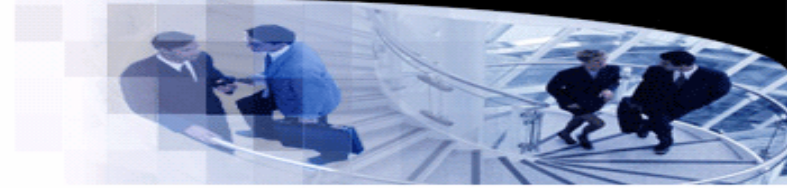
Lecture 12. Polymorphism

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

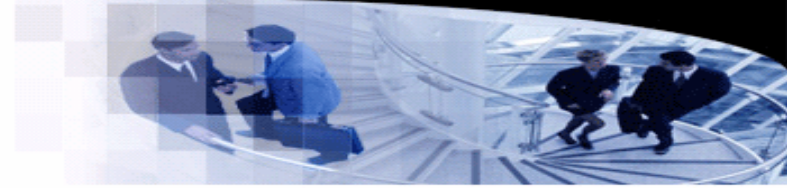
Introduction



- Varieties of polymorphism

- Type systems that allow a single piece of code to be used with multiple types are collectively known as **polymorphic systems**.
- Parametric polymorphism
 - It allows a single piece of code to be typed “generically,” using variables in place of actual types, and then instantiated with particular types as needed.
 - Parametric definitions are uniform: all of their instances behave the same.
 - **Let-polymorphism** restricts polymorphism to top-level let-bindings, disallowing functions that take polymorphic values as arguments.
 - **First-class polymorphism**, on the other hand, allows polymorphic values being passed as arguments.
 - First-class polymorphism forms the technical foundation for languages like ML.

Introduction

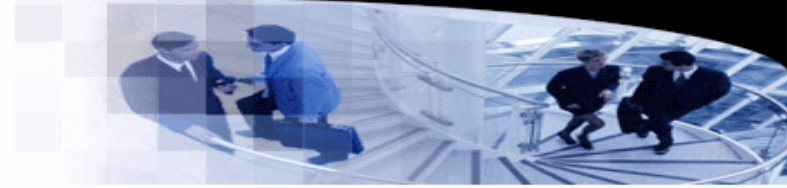


- Varieties of polymorphism

- Ad-hoc polymorphism

- It allows a polymorphic value to exhibit different behaviors when “viewed” at different types.
 - The most common example of ad-hoc polymorphism is **overloading**, which associates a single function symbol with many implementations.
 - The compiler (or the runtime system, depending on whether overloading resolution is static or dynamic) chooses an appropriate implementation for each application of the function, based on the types of the arguments.
 - **Intensional polymorphism** permits restricted computation over types at run time.
 - **Typecase primitive** permits arbitrary pattern-matching on type information at run time. Language features such as Java’s `instanceof` test can be viewed as restricted forms of typecase.

Type Reconstruction



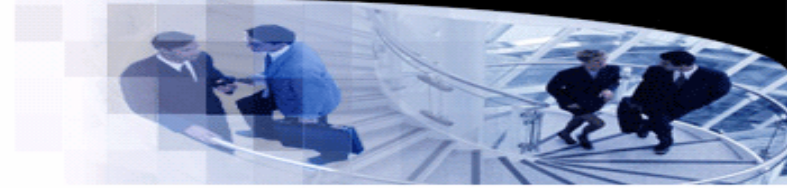
- **Type checking**

- Type checking algorithms rely on type explicit type annotations.
- We need a new approach to inferring types with less type information.

- **Type reconstruction**

- An algorithm that calculate a principal type for a term in which some or all of these annotations are left unspecified.
- Related algorithms play an important role in languages like ML and Haskell.
- Combining type reconstruction with other language features , subtyping for example, is quite challenging.
- We discuss type reconstruction for simple types in this case.

Type Reconstruction



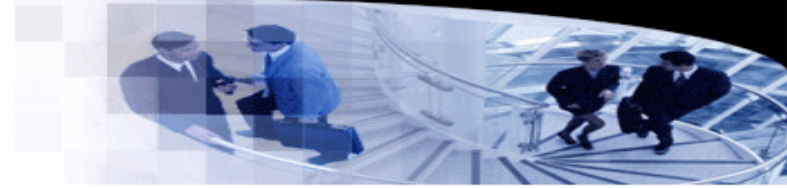
- **Type variables**

- We use **type variables**, which are placeholders, to represent uninterpreted base types.
- Type variables can be **substituted** or **instantiated** with other types.

- **Type substitution**

- A **type substitution** σ is a mapping from type variables to types. σ can be applied to a type T to obtain an instance σT .
- Example:
 - Let $\sigma = [X \mapsto \text{Bool}]$.
 - Apply σ to the type $X \rightarrow X$ to obtain $\sigma(X \rightarrow X) = \text{Bool} \rightarrow \text{Bool}$.

Type Reconstruction



- **Type substitution**

- **Definition:** A type substitution is a finite mapping from type variables to types.
- Example: $[X \mapsto T, Y \mapsto U]$ is the substitution that associates X with T and Y with U .
- **Application** of a substitution to a type is defined as follows.

$$\sigma(X) = \begin{cases} T & \text{if } (X \mapsto T) \in \sigma \\ X & \text{if } X \text{ is not in the domain of } \sigma \end{cases}$$

$$\sigma(\text{Nat}) = \text{Nat}$$

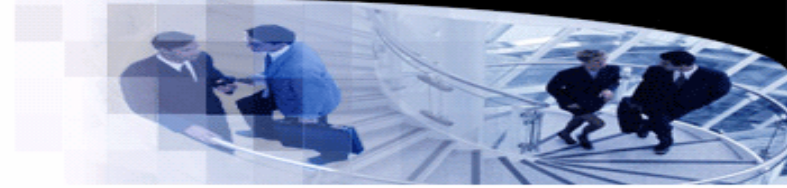
$$\sigma(\text{Bool}) = \text{Bool}$$

$$\sigma(T_1 \rightarrow T_2) = \sigma T_1 \rightarrow \sigma T_2$$

$$\sigma(x_1 : T_1, \dots, x_n : T_n) = (x_1 : \sigma T_1, \dots, x_n : \sigma T_n)$$

$$\sigma \circ \gamma = \left[\begin{array}{ll} X \mapsto \sigma(T) & \text{for each } (X \mapsto T) \in \gamma \\ X \mapsto T & \text{for each } (X \mapsto T) \in \sigma \text{ with } X \notin \text{dom}(\gamma) \end{array} \right]$$

Type Reconstruction



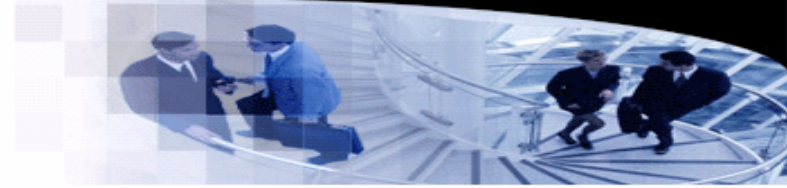
- **Type solutions**

- **Definition:** Let Γ be a context and t be a term. A solution for (Γ, t) is a pair (σ, T) such that $\sigma\Gamma \vdash \sigma t : T$.
- Example: Let $\Gamma = f : X, a : Y$ and $t = f\ a$. Then

$([X \mapsto Y \rightarrow \text{Nat}], \text{Nat})$ $([X \mapsto Y \rightarrow Z], Z)$
 $([X \mapsto Y \rightarrow Z, Z \mapsto \text{Nat}], Z)$ $([X \mapsto Y \rightarrow \text{Nat} \rightarrow \text{Nat}], \text{Nat} \rightarrow \text{Nat})$
 $([X \mapsto \text{Nat} \rightarrow \text{Nat}, Y \mapsto \text{Nat}], \text{Nat} \rightarrow \text{Nat})$

- are all solutions for (Γ, t) .

Type Reconstruction

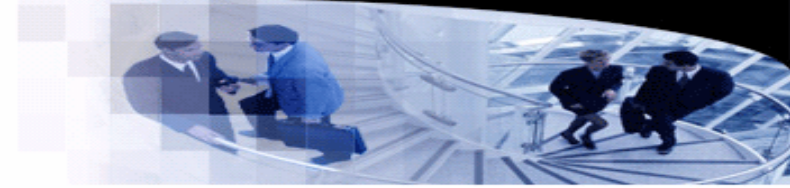


- **Constraint-based Typing**

- Given a term t and a context Γ , the typing algorithm calculates a set of **constraints** that must be satisfied by any solution for (Γ, t) .
- A constraint is an equation between type expressions.
- Example:
 - Given an application $t_1 \ t_2$ with $\Gamma \vdash t_1 : T_1$ and $\Gamma \vdash t_2 : T_2$, the type constraint for this application can be $T_1 = T_2 \rightarrow X$, where X is a type variable.
- **Definition:** A **constraint set** C is a set of equations $\{S_i = T_i \mid i \in 1..n\}$.
- **Definition:** The constraint typing relation $\Gamma \vdash t : T \mid_{\chi} C$ denotes that term t has type T under assumptions Γ whenever constraints C are satisfied. χ records the set of type variables in T .

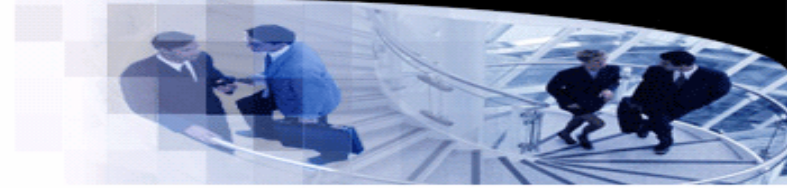
Type Reconstruction

- Constraint-based Typing



$\frac{x:T \in \Gamma}{\Gamma \vdash x : T \mid \emptyset \{ \}} \quad (\text{CT-VAR})$	
$\frac{\Gamma, x:T_1 \vdash t_2 : T_2 \mid_X C}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2 \mid_X C} \quad (\text{CT-ABS})$	
$\frac{\begin{array}{l} \Gamma \vdash t_1 : T_1 \mid_{X_1} C_1 \quad \Gamma \vdash t_2 : T_2 \mid_{X_2} C_2 \\ X_1 \cap X_2 = X_1 \cap FV(T_2) = X_2 \cap FV(T_1) = \emptyset \\ X \notin X_1, X_2, T_1, T_2, C_1, C_2, \Gamma, t_1, \text{ or } t_2 \\ C' = C_1 \cup C_2 \cup \{T_1 = T_2 \rightarrow X\} \end{array}}{\Gamma \vdash t_1 t_2 : X \mid_{X_1 \cup X_2 \cup \{X\}} C'} \quad (\text{CT-APP})$	
$\Gamma \vdash 0 : \text{Nat} \mid \emptyset \{ \} \quad (\text{CT-ZERO})$	
$\frac{\begin{array}{l} \Gamma \vdash t_1 : T \mid_X C \\ C' = C \cup \{T = \text{Nat}\} \end{array}}{\Gamma \vdash \text{succ } t_1 : \text{Nat} \mid_X C'} \quad (\text{CT-SUCC})$	
	$\frac{\begin{array}{l} \Gamma \vdash t_1 : T \mid_X C \\ C' = C \cup \{T = \text{Nat}\} \end{array}}{\Gamma \vdash \text{pred } t_1 : \text{Nat} \mid_X C'} \quad (\text{CT-PRED})$
	$\frac{\begin{array}{l} \Gamma \vdash t_1 : T \mid_X C \\ C' = C \cup \{T = \text{Nat}\} \end{array}}{\Gamma \vdash \text{iszero } t_1 : \text{Bool} \mid_X C'} \quad (\text{CT-ISZERO})$
	$\Gamma \vdash \text{true} : \text{Bool} \mid \emptyset \{ \} \quad (\text{CT-TRUE})$
	$\Gamma \vdash \text{false} : \text{Bool} \mid \emptyset \{ \} \quad (\text{CT-FALSE})$
	$\frac{\begin{array}{l} \Gamma \vdash t_1 : T_1 \mid_{X_1} C_1 \\ \Gamma \vdash t_2 : T_2 \mid_{X_2} C_2 \quad \Gamma \vdash t_3 : T_3 \mid_{X_3} C_3 \\ X_1, X_2, X_3 \text{ nonoverlapping} \\ C' = C_1 \cup C_2 \cup C_3 \cup \{T_1 = \text{Bool}, T_2 = T_3\} \end{array}}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T_2 \mid_{X_1 \cup X_2 \cup X_3} C'} \quad (\text{CT-IF})$

Type Reconstruction

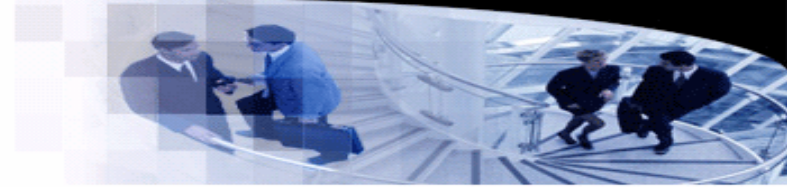


- The unification algorithm

- Definition: Suppose that $\Gamma \vdash t : S \mid C$. A solution for (Γ, t, S, C) is a pair (σ, T) such that σ satisfies C and $\sigma S = T$.
- The unification algorithm is based on the idea of using unifications by Hindley and Milner.

```
unify(C)  =  if C =  $\emptyset$ , then []  
            else let {S = T}  $\cup$  C' = C in  
              if S = T  
                then unify(C')  
              else if S = X and X  $\notin$  FV(T)  
                then unify([X  $\mapsto$  T]C')  $\circ$  [X  $\mapsto$  T]  
              else if T = X and X  $\notin$  FV(S)  
                then unify([X  $\mapsto$  S]C')  $\circ$  [X  $\mapsto$  S]  
              else if S = S1  $\rightarrow$  S2 and T = T1  $\rightarrow$  T2  
                then unify(C'  $\cup$  {S1 = T1, S2 = T2})  
              else  
                fail
```

Universal Types



- **System F**

- Motivation: We need infinite number of “doubling” functions for infinite argument types.

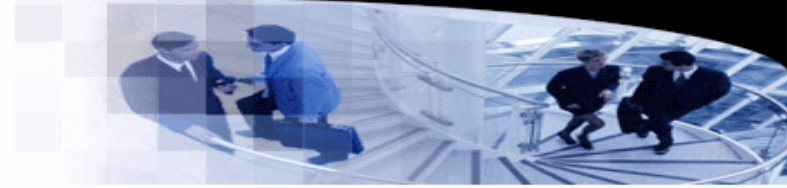
`doubleNat = λf:Nat→Nat. λx:Nat. f (f x);`

`doubleRcd = λf:{1:Bool}→{1:Bool}. λx:{1:Bool}. f (f x);`

`doubleFun = λf:(Nat→Nat)→(Nat→Nat). λx:Nat→Nat. f (f x);`

- It violates a basic dictum of software engineering.
 - **Abstraction principle:** Each significant piece of functionality in a program should be implemented in just one place in the source code. Where similar functions are carried out by distinct pieces of code, it is generally beneficial to combine them into one by abstracting out the varying parts.
- The varying parts are the types.
- We need facilities for abstracting out a type from a term and later instantiating this abstract term with concrete type annotations.

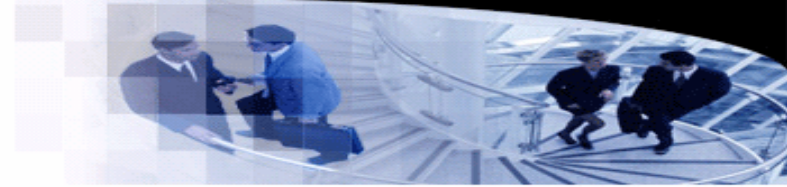
Universal Types



- **System F**

- First discovered by Jean-Yves Girard (1972), in the context of proof theory in logic.
- John Reynolds developed a type system and called it **polymorphic lambda-calculus**.
- It is also sometimes called the second-order lambda-calculus, because it corresponds, via the **Curry-Howard correspondence**, to second-order intuitionistic logic, which allows quantification not only over individuals [terms], but also over predicates [types].
- The definition is a straightforward extension of simply typed lambda calculus.
- A new form of abstraction, $\lambda x . t$, whose parameter is a type.
- A new form of application, $t \ [T]$, in which the argument is a type expression.

Universal Types



- System F

- New evaluation rules:

$$(\lambda X. t_{12}) [T_2] \longrightarrow [X \mapsto T_2] t_{12} \quad (\text{E-TAPP}TABS)$$

$$(\lambda x:T_{11}. t_{12}) v_2 \longrightarrow [x \mapsto v_2] t_{12} \quad (\text{E-APP}ABS)$$

- Example:

- Let the polymorphic identity function $id = \lambda X. \lambda x:X. x$.
 - We apply Nat to id by writing $id [Nat]$.
 - The result is $[X \mapsto Nat] (\lambda x:X. x) = \lambda x:Nat. x$.
 - This is the identity function on natural numbers.

Universal Types

- System F

$\rightarrow \forall$

Based on λ_- (9-1)

Syntax

$t ::=$

x

$\lambda x:T. t$

$t t$

$\lambda X. t$

$t [T]$

terms:

variable

abstraction

application

type abstraction

type application

$v ::=$

$\lambda x:T. t$

$\lambda X. t$

values:

abstraction value

type abstraction value

$T ::=$

X

$T \rightarrow T$

$\forall X. T$

types:

type variable

type of functions

universal type

$\Gamma ::=$

$\emptyset$

$\Gamma, x:T$

Γ, X

contexts:

empty context

term variable binding

type variable binding

Evaluation

$t \rightarrow t'$

$$\frac{t_1 \rightarrow t'_1}{t_1 t_2 \rightarrow t'_1 t_2}$$

(E-APP1)

$$\frac{t_2 \rightarrow t'_2}{v_1 t_2 \rightarrow v_1 t'_2}$$

(E-APP2)

$$(\lambda x:T_{11}. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12} \text{ (E-APPABS)}$$

$$\frac{t_1 \rightarrow t'_1}{t_1 [T_2] \rightarrow t'_1 [T_2]}$$

(E-TAPP)

$$(\lambda X. t_{12}) [T_2] \rightarrow [X \mapsto T_2] t_{12} \text{ (E-TAPPTABS)}$$

Typing

$\Gamma \vdash t : T$

$$\frac{x:T \in \Gamma}{\Gamma \vdash x : T}$$

(T-VAR)

$$\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$$

(T-ABS)

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}}$$

(T-APP)

$$\frac{\Gamma, X \vdash t_2 : T_2}{\Gamma \vdash \lambda X. t_2 : \forall X. T_2}$$

(T-TABS)

$$\frac{\Gamma \vdash t_1 : \forall X. T_{12}}{\Gamma \vdash t_1 [T_2] : [X \mapsto T_2] T_{12}}$$

(T-TAPP)

Universal Types

- System F

- Examples:

- Polymorphic doubling function.

```
double = λX. λf:X→X. λa:X. f (f a);      double [Nat] (λx:Nat. succ(succ(x))) 3;
```

► $\text{double} : \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$

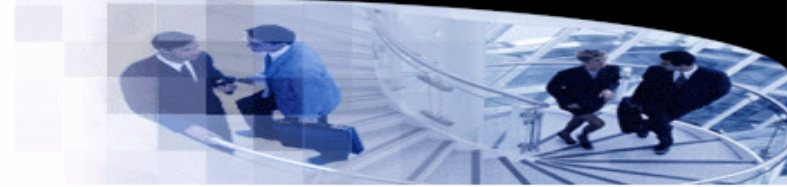
► $7 : \text{Nat}$

```
doubleNat = double [Nat];
```

► $\text{doubleNat} : (\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}$

```
doubleNatArrowNat = double [Nat→Nat];
```

► $\text{doubleNatArrowNat} : ((\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}) \rightarrow$
 $(\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}$



Universal Types

- System F

- Examples:

- Church encodings.
- The original church encodings are untyped.

$\text{tru} = \lambda t. \lambda f. t;$

$\text{fls} = \lambda t. \lambda f. f;$

- The polymorphic version in system F.

$\text{CBool} = \forall X. X \rightarrow X \rightarrow X;$

$\text{tru} = \lambda X. \lambda t:X. \lambda f:X. t;$

► $\text{tru} : \text{CBool}$

$\text{fls} = \lambda X. \lambda t:X. \lambda f:X. f;$

► $\text{fls} : \text{CBool}$

$\text{not} = \lambda b:\text{CBool}. \lambda X. \lambda t:X. \lambda f:X. b [X] f t;$

► $\text{not} : \text{CBool} \rightarrow \text{CBool}$

